

NeXTA-X Users Guide

Cross-platform AMS visualization for DTALite, TAPLite, Dynamic ODME, and DLSim

2026-07-02

Contents

1. What is NeXTA-X	1
2. Getting started	2
3. Loading data (unified GMNS reading)	2
4. Layer Control Panel	2
5. MOE Toolbar	2
6. Animation View	2
7. Inspector and Summary Statistics	3
8. Live-follow and running the engine	3
9. Sample datasets (datasets/)	3
10. Troubleshooting	3

1. What is NeXTA-X

NeXTA-X is the cross-platform successor of the NEXTA GUI for viewing AMS (Analysis–Modeling–Simulation) assignment and simulation results. It follows the classic NEXTA structure — **Layer Control Panel**, **MOE Toolbar**, **Animation View**, link inspector, summary statistics — re-built on one shared data contract (`SHARED_CONTRACT.md`) with three viewers for three situations:

branch	file	when to use
web-lite	nexta_x.html	zero-install QC: open in any browser, drag-drop files; ships inside a run folder
web-gl	web-gl/nexta_xgl.html	GPU animation at regional scale (100k+ vehicles) and live-follow of a running simulation
desktop-qt	desktop-qt/nexta_qt.py	desktop app: open folders, Run engine button (launches DLSim and reloads results), live-follow

Network **editing** is intentionally not reimplemented — use the legacy NEXTA Windows exe for editing; NeXTA-X reads the same GMNS files afterward.

2. Getting started

web-lite — double-click `nexta_x.html`, then either drag-drop your files onto the map or serve a folder:
`python -m http.server 8765 -> http://localhost:8765/nexta_x.html?data=demo`.

web-gl — same, page is `web-gl/nexta_xgl.html`; demo links: `?data=../demo` (Chicago Sketch, 30k vehicles) and `?data=demo_regional` (Chicago Regional, 20k vehicles).

desktop-qt — `pip install PySide6`, then:

```
python desktop-qt/nexta_qt.py datasets/02_chicago_sketch           # open a dataset
python desktop-qt/nexta_qt.py <folder> --snapshot out.png          # headless render (CI / reports)
```

3. Loading data (unified GMNS reading)

Every branch reads the same files; drop any subset — roles are detected from name + header:

file	gives you
<code>node.csv</code> , <code>link.csv</code>	the network (WKT geometry polylines used when present; zone centroids highlighted)
<code>link_performance.csv</code>	link volume / speed / max queue -> MOE coloring, VMT, top-congested list
<code>link_performance_15min.csv</code>	time-dependent MOE -> animation bins
<code>path_flow.csv</code> / <code>route_assignment.csv</code>	path layer (top OD paths, <code>link_ids</code> sequences)
<code>agent_trajectory.csv</code>	vehicle animation (event list (<code>agent</code> , <code>link</code> , <code>time</code> , <code>buffer</code> , <code>state</code>))
<code>run_summary.json</code> , <code>gridlock_events.csv</code>	run panel: conservation, VMT/VHT, oversaturation warnings

Engines: DTALite/TAPLite emit `link_performance.csv`/`route_assignment.csv` directly; DLSim emits the full package via `dlsim_run.exe <scenario> [--traj N] [--odme K]`; ODME comparisons load `measurement.csv` and `od_adjustment_with_sensor_coverage.csv` (overlays: roadmap P2).

4. Layer Control Panel

Toggle **nodes** (gold = zone centroids), **links**, **paths** (top-K OD paths in distinct colors), **vehicles** (trajectory dots). Same checkboxes in all branches.

5. MOE Toolbar

Select the link measure: **volume** (bandwidth + color), **V/C**, **queue**, or **td** (time-dependent — animates per 15-min bin). Color ramp: green (free) -> yellow -> red (congested); selected link is blue. Bandwidth scales with the measure, as in classic NEXTA.

6. Animation View

Press **Play** (or scrub the time slider). The clock advances in simulation minutes at the chosen speed; link colors follow the current 15-min bin and vehicles move along links: **green dot = moving** (position

interpolated between entrance- and exit-buffer times), **red dot = queued** at the link end; dots disappear on trip completion. web-gl performs this interpolation **in the GPU vertex shader** — 746k trajectory segments animate at full frame rate on the Chicago Regional network.

7. Inspector and Summary Statistics

Click any link: id, from->to nodes, volume, V/C, max queue, lanes/capacity/length, and the current bin's inflow/queue. The statistics panel shows nodes/zones/links, VMT, TD bin count, agents, top congested links, and the **run summary** — engine, conservation, VMT/VHT, and the **gridlock section** (oversaturated flag, first warning time, deadlock cycles, storage bypasses).

8. Live-follow and running the engine

- **web-gl**: tick *live-follow* — the page polls the served folder every 5 s and reloads any file that changed. Start a DLSim/DTALite run writing into that folder and watch it progress.
- **desktop-qt**: *Run engine* launches `dlsim_run.exe` on the open folder (engine log docked at the bottom) and reloads the outputs on completion; *live-follow* polls like the web version.

9. Sample datasets (datasets/)

dataset	scale	layers available
01_sioux_falls	24 nodes / 76 links	network, demand
02_chicago_sketch	933 nodes / 2,950 links	all : MOE, 26 TD bins, 250 paths, 30k vehicles, run+gridlock
03_chicago_regional	12,982 nodes / 39,018 links	network, MOE, 20k vehicles, run summary
04_arc_atlanta	66k nodes / 145,971 links	network + real DTALite assigned volumes (MOE)
05_toy_bottleneck / merge / signal	3-5 nodes	teaching: 15-min TD animation of queue formation/discharge

All are genuine model outputs (DLSim runs; ARC volumes from the DTALite equilibrium run), not synthetic.

10. Troubleshooting

- **Blank map** -> load `node.csv` + `link.csv` first (or together with outputs); press *Fit*.
- **No animation** -> needs `link_performance_15min.csv` (bins) and/or `agent_trajectory.csv` (vehicles); generate with `dlsim_run.exe <scenario> --traj 2000`.
- **Huge networks in web-lite** -> use web-gl (GPU) or desktop-qt; web-lite targets $\leq \sim 50k$ links.
- **Trajectories are privacy level 0** — export only via the explicit `--traj` opt-in; the `privacy_manifest.json` in each run package records what was exported.
- **desktop-qt offscreen snapshots** show placeholder glyphs for text on some systems (font-less offscreen platform); on-screen rendering is unaffected.